11/02/26

Network Security 2, Rubric 2

Advanced Network Architecture
The project must include implementation of basic network segmentation using VLANs in a virtual lab environment. Zero Trust concepts must be demonstrated through at least one practical example (such as user verification before resource access). Basic secure routing configuration must be demonstrated with evidence of security settings. A simple security workflow must be created that connects at least two security tools. All implementations must include screenshots and configuration notes.

1. Lab Architecture & Interface Mapping

Set up your virtual networking (using LAN Segments in VMware, Internal Networks in VirtualBox, or Bridges in Proxmox) so OPNsense dictates the boundaries.

- **OPNsense WAN Interface:** Connected to your home internet (NAT/Bridged).
- **OPNsense LAN Interface (Physical/Parent NIC):** Used as the trunk for virtual tags.
- **VLAN 10 (Corporate Users):** IP `192.168.10.1/24` → **Ubuntu VM** (`192.168.10.50`)
- **VLAN 20 (Secure Server Zone):** IP `192.168.20.1/24` → **Target Application VM** (`192.168.20.100`)
- **VLAN 99 (Attacker/SOC Zone):** IP `192.168.99.1/24` → **Kali Linux VM** (`192.168.99.15`)

---

2. Step-by-Step Implementation & Evidence Guide

🔹 **Component 1: Basic Network Segmentation (VLANs in OPNsense)**

Goal: Restrict physical broadcast domains by separating your Ubuntu machine, your Server, and your Kali machine at Layer 2.

- **Execution Steps:**
    1. Log into OPNsense → Navigate to **Interfaces > Other Types > VLAN**.
    2. Click **Add**. Set **Tag:** `10`, **Parent Interface:** (Select your LAN interface, e.g., `vtnet1`), **Description:** `Corp_LAN`. Click Save.
    3. Repeat to create **VLAN 20** (`Server_Zone`) and **VLAN 99** (`Kali_Zone`).
    4. Navigate to **Interfaces > Assignments**. Under **Assign a new interface**, map your new VLANs, click **Add**, then click the interface name to check **Enable Interface** and assign a Static IPv4 address (e.g., `192.168.10.1` for VLAN 10).
- **Configuration Note Template:**
  text

```
None


OPNsense VLAN Configuration Table:
-----------------------------------------------------------
Device      | Tag | Subnet           | Assigned IP
-----------------------------------------------------------
Corp_LAN    | 10  | 192.168.10.0/24  | 192.168.10.1
Server_Zone | 20  | 192.168.20.0/24  | 192.168.20.1
Kali_Zone   | 99  | 192.168.99.0/24  | 192.168.99.1
```

- Use code with caution.
- 📸 **Screenshot Focus:** The **Interfaces > Assignments** page showing the VLAN tags mapped cleanly to their virtual device names.

🔹 **Component 2: Basic Secure Routing (OPNsense Firewall Rules)**

Goal: Prevent lateral movement between VLANs. By default, OPNsense blocks everything. You will explicitly allow the Ubuntu machine to talk to the web server over a secure port, while completely blocking the Kali machine from touching either subnet.

- **Execution Steps:**

1. Go to **Firewall > Rules > Corp_LAN (VLAN 10)**.
2. Add a rule: **Action:** Pass | **Protocol:** TCP | **Source:** `Corp_LAN net` | **Destination:** Single Host `192.168.20.100` | **Destination Port:** `80` or `443` (your web application).
3. Go to **Firewall > Rules > Kali_Zone (VLAN 99)**. Ensure there are **no** pass rules, ensuring the default block takes effect.

- **Configuration Note Template:**
  text

None

```
Firewall Action Policy (Rule Order top-to-bottom):
1. PASS TCP | Source: VLAN10_net | Dest: 192.168.20.100 Port 443 |
Desc: Allow Intranet Web
2. BLOCK ALL | Source: * | Dest: * | Desc: Default Implicit Deny
```

- Use code with caution.
- 📸 **Screenshot Focus:**
  1. Your **Firewall > Rules > Corp_LAN** interface showing the explicit allow rule.
  2. A split terminal screen: On **Ubuntu**, a successful `curl http://192.168.20.100` connection; on **Kali**, a timed-out ping or connection attempt to the same IP.

🔹 Component 3: Zero Trust Architecture (User Verification)

Goal: Enforce authentication before exposing a backend asset. You will host a simple application on your Server (VLAN 20) behind a reverse proxy that forces access via an identity provider before passing traffic.

- **Practical Setup:** Use Docker on your Ubuntu/Server VM to spin up a quick application protected by a Zero Trust portal like **Authelia** or **Authentik**.
- **Execution Steps:**
  1. User requests access to `http://192.168.20.100`.
  2. The routing table successfully delivers the packet, but the application interceptor steps in: **"Never trust, always verify."**

3. The browser is instantly hijacked and redirected to a centralized authentication page requiring standard credentials plus an MFA prompt (TOTP).

- **Configuration Note Template:**
  yaml

```
None


# Zero Trust Interceptor Rule Configuration
server {
    listen 443 ssl;
    server_name secure-app.local;
    # Redirect to Zero Trust Verification Portal
    auth_request /internal/auth/verify;
    error_page 401 = @clean_auth_redirect;
}
```

- Use code with caution.
- 📸 **Screenshot Focus:** A browser session running inside your VLAN 10 Ubuntu VM showing the exact moment it hits the server's IP address but is halted by an enforcement login wall requesting a username, password, and token.

🔹 **Component 4: Simple Security Workflow (OPNsense Crowdsourced Blocking / IDS to Firewall)**

Goal: Connect two security tools seamlessly. We will connect the built-in **Suricata IDS** engine inside OPNsense to the **OPNsense Firewall Packet Filter (pf)** rule execution tool.

- **The Automated Workflow:**
  $\text{Kali\ Machine\ (VLAN\ 99\ Scan)}\longrightarrow \text{Tool\ 1:\ Suricata\ (Detects\ Scan)}\longrightarrow \text{Automation\ Engine}\longrightarrow \text{Tool\ 2:\ Firewall\ Table\ (Instantly\ Drops\ Target\ IP)}$
- **Execution Steps:**
  1. Navigate to **Services > Intrusion Detection > Administration**.
  2. On the **Settings** tab, check **Enable**, select the **Kali_Zone (VLAN 99)** interface, and check **IPS mode** (Intrusion Prevention).

3. Go to the **Download** tab, enable standard scanning rulesets (like *ET Scan*), and click **Download & Update**.
4. Under **Rules**, find scan signatures (e.g., Nmap scans) and switch their action from *Alert* to *Drop*.

- 📸 **Screenshot Focus:**
  1. Fire up your **Kali VM** and execute a basic port scan: `nmap -sV 192.168.20.100`.
  2. Take a screenshot of the **Services > Intrusion Detection > Alerts** screen showing Suricata catching the Kali IP address red-handed.
  3. Take a screenshot of the firewall logs showing that the attack traffic dropped mid-scan due to automated rule orchestration.
